

中原大學 111 學年度

☐ 上學期
☒ 下學期

考試命題紙

☐ 期中
☒ 學期 考試

科目名稱：程式語言

考試時間：6 月 13 日第 7 節

開課班級：資訊三甲/資訊三乙

課程代碼：CS220G/CS220H

【考試要認真 成績靠自己】

教師簽名：賴建宏

* ☒ 皆不可帶 ☐ 可攜帶.....應考

* ☐ 需要答案紙 ☒ 不需要答案紙(直接於命題紙上作答)

本份試題共 8 頁，本版為第 1 頁

(請勿在背面命題)

班級：_____ 學號：_____ 姓名：_____

1. 答案是直接寫在此試卷(也稱為‘命題紙’)上，而不是另用答案卷來寫。
2. **用中文回答！**(但專有名詞或 keywords 可以是英文，也該用英文，以免有誤解)
3. 解釋事情時不要「言簡意賅」或是「惜字如金」；因為「說得模糊不清或模稜兩可或不知所云」=「沒說」！

1. (14 pts.) 下面列出一個文法、以及「此文法中出現的 token」與 lexeme 的關係

token

lexeme

ID	(any legal identifier, e.g., "abc", "sum25")
COMMA	,
SEMICOLON	;
LEFT_BRACKET	[
RIGHT_BRACKET	]
LEFT_BRACE	{
RIGHT_BRACE	}
CONST	(一個正整數, e.g., 12, 25)
INT	int
FLOAT	float
CHAR	char
TYPEDEF	typedef
STRUCT	struct

<variable declaration> ::= <type> ID { COMMA ID } SEMICOLON

<type> ::= INT | FLOAT | CHAR | NEW_TYPE

NEW_TYPE 是任何符合以下(1)或(2)或(3)寫法的 ID：

- (1) TYPEDEF <type> [*] ID SEMICOLON
- (2) TYPEDEF <type> ID LEFT_BRACKET CONST RIGHT_BRACKET SEMICOLON
- (3) STRUCT ID LEFT_BRACE <variable declaration> { <variable declaration> } RIGHT_BRACE SEMICOLON

請問為什麼此種「宣告變數」的設計符合 orthogonal design principle？

(a) (8 pts.) 請依 orthogonal design principle 的四個要項分別闡述之。

◎ 我們瞭解人人各承不同之稟賦，其性格、能力與環境各異，故充分發揮個人潛力就是成功。~ 我校教育理念

請勿在此線以下命題

科目名稱：程式語言

開課班級：資訊三甲/三乙

本份試題共 8 頁，本版為第 2 頁

考試時間：6 月 13 日第 7 節

課程代碼：CS220G/CS220H

(b) (6 pts.) 你也必須舉出一個相當完整的「宣告某個變數的例子」、用此例子來(進一步)說明之。

(a)

Primitive data type 不多：因為只有四種基本數據類型：INT，FLOAT，CHAR，和 NEW_TYPE。這使得學習和使用此種語言變得容易。

Combine 的方式不多：可以透過三種方式建立新的資料型別 (TYPEDEF，TYPEDEF with CONST 或 STRUCT)；其次在變數宣告上可以透過逗號將多個宣告合併在同一行，或是各自獨立一行。由此可知此種語言的 Combine 方式不多，但使得 combine 變得簡單使用。

怎樣 combine 都行：可以自由 combine 基本資料型別以建立新的資料型別或是宣告各式變數。

任意複雜度都可組合：可以建立一個新的結構，其結構包含其他型別，或者建立一個指向該結構的指標。。

請勿在此線以外命題

(b)

```
STRUCT PLStruct LEFT_BRACE
    INT x SEMICOLON
    FLOAT y SEMICOLON
RIGHT_BRACE SEMICOLON
```

//定義了一個叫做 PLStruct 的結構，裏頭包含一個整數型別的變數 x 和一個浮點數型別的變數 y

//然後，用此結構宣告變數

```
PLStruct PL1 SEMICOLON
```

//宣告一個叫做 PL1 的結構變數

以上程式碼即遵循了 orthogonal design principle，因為可以獨立使用 INT 和 FLOAT 定義結構，並且也使用該結構獨立地宣告變數，表示此種語言設計可以簡單定義和使用任意複雜的資料型別。

科目名稱：程式語言

開課班級：資訊三甲/三乙

本份試題共 8 頁，本版為第 2 頁

考試時間：6 月 13 日第 7 節

課程代碼：CS220G/CS220H

(b) (6 pts.) 你也必須舉出一個相當完整的「宣告某個變數的例子」、用此例子來(進一步)說明之。

(a)

Primitive data type 不多：因為只有四種基本數據類型：INT，FLOAT，CHAR，和 NEW_TYPE。這使得學習和使用此種語言變得容易。

Combine 的方式不多：可以透過三種方式建立新的資料型別 (TYPEDEF，TYPEDEF with CONST 或 STRUCT)；其次在變數宣告上可以透過逗號將多個宣告合併在同一行，或是各自獨立一行。由此可知此種語言的 Combine 方式不多，但使得 combine 變得簡單使用。

怎樣 combine 都行：可以自由 combine 基本資料型別以建立新的資料型別或是宣告各式變數。

任意複雜度都可組合：可以建立一個新的結構，其結構包含其他型別，或者建立一個指向該結構的指標。。

(b)

STRUCT PLStruct LEFT_BRACE

INT x SEMICOLON

FLOAT y SEMICOLON

RIGHT_BRACE SEMICOLON

//定義了一個叫做 PLStruct 的結構，裏頭包含一個整數型別的變數 x 和一個浮點數型別的變數 y

//然後，用此結構宣告變數

PLStruct PL1 SEMICOLON

//宣告一個叫做 PL1 的結構變數

以上程式碼即遵循了 orthogonal design principle，因為可以獨立使用 INT 和 FLOAT 定義結構，並且也使用該結構獨立地宣告變數，表示此種語言設計可以簡單定義和使用任意複雜的資料型別。

科目名稱：程式語言

開課班級：資訊三甲/三乙

本份試題共 8 頁，本版為第 3 頁

考試時間：6 月 13 日第 7 節

課程代碼：CS220G/CS220H

2. (15 pts.) 請參考以下的 C++ 程式與 Java 程式，然後寫出各個變數是在哪裡(是在 call stack 裡、還是在 heap 裡?)。(例：x 是在 call stack 中，能存一個 Student struct；會有個無名變數在 heap 中，能存 15 個 double；y 是在 call stack 中，能存 15 個 pointer；會有個無名變數在 heap 中，能存一個 int；)

// 我們假設 'Student' 是個已宣告好的 (Java 或 C++) class。

// 我們也假設以下這些宣告是出現於某個 function 的 function body 之中。

// 注意：你的答案要讓批改者看得出來是針對哪個宣告所寫的答案，避免錯失分數。

C++：

// 請把你的答案直接寫在各個宣告的右邊。

Line 1: Student s1; // s1 在 call stack 上，Student Type 的 instance

Line 2: Student s2[20]; // s2 在 call stack 上，可以儲存 20 個 Student instance

Line 3: Student *s3 = new Student; // s3 在 call stack 上，是個指向 Student 型別的 pointer。被指向的 student instance 是在 heap

Line 4: int a; // a 在 call stack 上，是一個整數變數。

Line 5: char b[20]; // b 在 call stack 上，可以儲存 20 個字元的陣列。

Line 6: char *c = new char[20]; // c 在 call stack 上，指向字元型別的 pointer。被指向的是可儲存 20 個字元的陣列且在 heap 上。

Java：

// 請把你的答案直接寫在各個宣告的右邊。

Line 7: Student s4 = new Student(); // s4 在 call stack 上，是個 reference(參考)，用來指向 Student object。生成的 Student object 是在 heap 中。

Line 8: Student[] s5 = new Student[20]; // s5 在 call stack 上，是個 reference(參考)，用來指向 Student object 的陣列。生成的 Student object 的陣列則在 heap 中，而陣列中的每個元素則是 reference(參考)，且都是用來指向 Student object，所生成的 object 也都是在 heap 中。

Line 9: int d; // d 在 call stack 上，是整數型別的變數。

Line 10: char[] e = new char[20]; // e 在 call stack 上，是個 reference(參考)，用來指向字元陣列。生成的字元陣列則是在 heap 上。

科目名稱：程式語言

開課班級：資訊三甲/三乙

本份試題共 8 頁，本版面為第 4 頁

考試時間：6 月 13 日第 7 節

課程代碼：CS220G/CS220H

3. (10 pts.) 教科書上有下面這麼一句話，請問為什麼這樣說？請儘可能解釋清楚！

Coercions can result in a loss of one of the benefits of strong typing - error detection.

Strong typing 意指在 compile 時檢查型別的正确性，如果型別不吻合，如將整數型別的變數 assign 給字元型別的變數，此時即會產生錯誤。此做法有助於在程式執行之前就發現並修正錯誤。然而，當 Compiler 使用 coercions 時，透過改變變數的型別，使之能符合需求（如上述例子），則就有可能規避 compiler 的 type checking，如此即有可能在 runtime 時發生錯誤，也就失去了 strong typing 在 error detection 上的優勢。

4. (10 pts.) 請分析 Static Type Binding 和 Dynamic Type Binding 的差異，並舉例說明。

Static Type Binding：在 Static Type Binding 中，變數的型別在 compile 就被確定，並且該變數在消失（生命週期, lifetime）以前是不會改變型別。也因為在 compile 時期就確定型別，所以可以在 compile 時檢查型別的正确性，如果型別不吻合，就會產生錯誤。優點即是提前發現型別問題並修正錯誤，避免在 runtime 時出現錯誤。例如：int a; 表示宣告一個整數型別的變數

- explicit declaration：在一個 statement 中列出變數的名稱，並宣告它們所屬的型別。
- implicit declaration：在程式中使用者沒有宣告過的變數，系統會依其內定的規則進行宣告。

■ 優點：對使用者較為方便，不用擔心變數忘記宣告

■ 缺點：降低程式的可靠度，因 compiler 難以偵測變數拼錯的問題；其次是對於不小心忘記宣告的變數，若系統內定的型別與使用者預期的不同，會導致 runtime 或 logical error。

Dynamic Type Binding：在 Dynamic Type Binding 中，變數的型別直到 runtime 時才被確定，並且變數在消失（生命週期, lifetime）以前是可以改變型別。優點即是讓型別使用有彈性，但相對缺點即是導致 runtime 時出現型別錯誤，因為這些錯誤在 compile 時都會被認定為正常，無法被檢測出來；其次是降低 compiler 的錯誤偵測能力，並增加實作成本，因為必須進行 runtime 時期的型別檢查，且每個變數在執行時都必須保留額外的空間用來記錄型別資訊。例如：

a = 1 # a 在這個時候是整數型別的變數

a = "hi" # a 在這個時候是字串型別的變數

科目名稱：程式語言

開課班級：資訊三甲/三乙

本份試題共 8 頁，本版為第 5 頁

考試時間：6 月 13 日第 7 節

課程代碼：CS220G/CS220H

5. (16 pts.) 談到了 Storage Binding，就不得不談一下 static variable、stack-dynamic variable、explicit heap-dynamic variable 以及 implicit heap-dynamic variable，請問這四個各自是什麼？請舉例說明，並分析其優缺點。

(1) static variable: 在程式執行前即已 binding 到記憶體，且一直到執行終止都不會改此種 binding 的變數。

以下皆為 static variable

甲、global variables: 在整個程式中都可以使用的變數

乙、history-sensitive variables // local variables: 即宣告在副程式中，且只有在該副程式內可以使用的 static local variable

優點：(1) 執行效率佳，因為 static variable 可以直接定址 (2) 執行時期不需處理記憶體之配置和釋放工作

缺點：(1) 記憶體的運用缺乏彈性，因為不管此種變數在執行時是否被用到，一定都會先配置記憶體給它 (2) 無法支援遞迴

(2) stack-dynamic variable: 變數的型別在 compile 時 binding，但其記憶體的 binding 則發生在所屬程式碼要執行時。例如 function 的區域變數。

優點：(1) 可以支援遞迴 (2) 較節省記憶體，副程式沒有執行時，local variable 不會占用記憶體

缺點：(1) 執行時期必須進行記憶體的配置與釋放 (2) 無法提供變數回顧的相關特性。

(3) explicit heap-dynamic variable: 經由執行指令如 new 而存在，是個沒有名稱的變數（記憶體空間），其記憶體在執行時才由程式所指定的命令配置或清除，此種變數（記憶體空間）只能經由指標來 access。變數型別的 binding 在 compile 時發生；記憶體的 binding 在變數被建立時發生(run time)。

優點：(1) 適合用來處理動態結構，如 tree, linked list。

缺點：(1) 較難正確使用 (2) 成本較高，必須間接引用，且必須自行配置和清除記憶體。

(4) implicit heap-dynamic variable: 經由 assign 而存在，如 str = "Hi" + ' ' + "ICE"，當變數被指定值時才 binding 記憶體。

優點：具有高度彈性

缺點：(1) 降低 compiler 錯誤偵測能力 (2) 增加執行時期之實作成本

科目名稱：程式語言

開課班級：資訊三甲/三乙

本份試題共 8 頁，本版為第 6 頁

考試時間：6 月 13 日第 7 節

課程代碼：CS220G/CS220H

6. (10 pts.) 指標(pointer)使用不當，可能引發的問題為何？請至少列舉兩個問題。而又為什麼 Java 在指標使用方面，比 C 語言較為安全呢？

- (1) Null Pointers：如果 pointer 被賦予 NULL 並且試圖透過該 pointer access 時，會導致 runtime error。
- (2) Dangling Pointers：當一個指標指向已經被釋放或未分配的記憶體空間時，即稱為 Dangling Pointers。也因為該記憶體空間被釋放，所以任何嘗試 access 該記憶體空間的操作都會導致不可預測的問題。
- (3) 未初始化的 pointers：由於指標的值未定義，可能指向任何記憶體地址，導致 runtime 時不可預測。

為何 Java 在指標使用上較為安全？

根據 Java 官網表示，其設計是為了讓語言能更安全，因此，Java 選擇不直接支援 pointer，但有 reference，表示所有的 object 都是透過 reference access，且 Java 可進行 garbage collection，如此即可避免前述指標的問題。而對於空指標，Java 有 NullPointerException，使得可以在開發階段時就發現問題。

7. (5 pts.) 在何種情況下，call by reference 和 call by value-result 會得到不同的結果？請舉例說明。

"Call by Reference" 和 "Call by Value-Result" 都允許 function 修改呼叫者的資料內容，差別在於何時更新呼叫者的資料內容。"Call by Reference" 是立即更新，因為是立即針對該記憶體位址內的值修改，而 "Call by Value-Result" 則是在 function 結束後才更新。因此對 call by reference 而言，當兩個 formal parameter 互為別名或 formal parameter 與非區域變數互為別名時，call by reference 和 call by value-result 可得到不同結果。原因是 call by reference 時會直接對有別名的 formal parameter 進行改變，同時也會影響到 actual parameter；然而 call by value-result 時，由於每一個 formal parameter 皆擁有各自的記憶體空間，過程之中每一個 formal parameter 的改變都是各自獨立改變，不會相互有影響，而且只有到回傳時才會對 actual parameter 造成影響。

```
int x = 10;
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void main() {
    swap(&x, &x);
    printf("%d\n", x);
}
```

科目名稱：程式語言

開課班級：資訊三甲/三乙

本份試題共 8 頁，本版面為第 7 頁

考試時間：6 月 13 日第 7 節

課程代碼：CS220G/CS220H

8. (5 pts.) 請說明 Overloaded 和 polymorphic subprogram 的主要不同之處。

Overloading: Overloading 表示是指多個 function 使用同一名稱，但是參數列表不同，如參數個數不同，參數型別不同或是回傳值不同等，如此可以根據參數的型別和個數，將相同名稱的函數用於不同的目的。例如 C++ 中的 cout。Overloading 是在 compile 時處理。

Polymorphic subprogram: 可視為一個 function 的模板，本身無法被執行，但是它將 subprogram 中某些屬性參數化，例如將資料型別以 formal parameter 表示，使用者即可透過產生 Instantiation 的方式產生特定的 subprogram。Instantiation 是由 compiler 自動執行，例如將前述的資料型別代入 formal parameter，以產生可執行的 subprogram。因此只要定義一個 Polymorphic subprogram，即可根據自己的需求透過 Instantiation 產生特定的 subprogram。

```
public static <T> T getPL(List<T> list) {
    return list.get(0);
}
```

// <T> 為形式參數，代表未知的型別
// 當在程式中呼叫 getPL 方法時可以將 T 替換為任何型別

這部分與 Polymorphism 的本意相同，延後 function binding 的時間等到執行時期才決定要呼叫哪一個 function。如此可使程式的執行不在 compile 時期決定，而延後到 runtime 期間才決定呼叫誰。

9. (15 pts.) Java 的記憶體管理方式採用 garbage collection:

(9a.) (6 pts.) Reference count 和 Mark-sweep 是 garbage collection 的作法，請舉例說明二者的作法。

Reference Counting: 追蹤和記錄每個 object 被引用的次數。

Mark-Sweep: 將整個記憶體被 access 到的進行標記，未被標記的就視為需要回收。

科目名稱：程式語言

開課班級：資訊三甲/三乙

本份試題共 8 頁，本版為第 8 頁

考試時間：6 月 13 日第 7 節

課程代碼：CS220G/CS220H

(9b.) (4 pts.) Java 基於何種理由採用 garbage collection ?

- (1) 允許使用者專注於功能開發而不需擔心記憶體管理問題，減少了使用者在手動釋放記憶體和 memory leak 等問題上的時間成本。
- (2) 提升程式的可靠性：透過自動化 garbage collection，減少錯誤的記憶體操作（如 dangling reference/pointer）所引發的錯誤。
- (3) 防止 memory leak
- (4) 簡化應用程式的記憶體管理

以上寫一點 +2
兩點 +4

(9c.) (5 pts.) 說明 garbage collection 的優缺點。

優點

- (1) 可避免因 dangling reference 而對系統造成危害
- (2) 當程式釋放記憶體時，因系統暫不處理，可提供執行效率

缺點

- (1) 進行 garbage collection 須暫停使用者之程式執行
- (2) garbage collection 工作複雜，且需花費大量時間

寫其它打接受優點 +1

其它寫到一點 +2.5